

Contractual Skills: A GovernSpec Design Framework for Enterprise AI Agents

Ting Liu
SymbolicLight Research
Foshan, Guangdong, China
research@symboliclight.com

May 2026

Abstract

Skills have become a practical packaging mechanism for agent instructions, workflows, scripts, and reference materials. In enterprise settings, however, a skill is often expected to do more than trigger relevant context. It must also express input boundaries, permissions, human approval points, evidence requirements, output contracts, quality criteria, verification steps, and handoff rules. When these controls are written as informal prose, they are difficult to review, reuse, test, or connect to runtime guardrails. This paper proposes *contractual skills*, a design framework that organizes `SKILL.md` files with GovernSpec-style task contract fields while preserving the lightweight discovery and progressive-loading properties of skills. The framework positions a skill as a readable task contract rather than as a free-form prompt fragment. It also clarifies the boundary between contractual skills, GovernSpec YAML contracts, Model Context Protocol (MCP) surfaces, tool adapters, runtime guardrails, tracing, and evaluation systems.

We evaluate the framework with two offline experiments. The text-generation experiment covers three enterprise skills, fifteen synthetic tasks, four instruction conditions, and eight generation models, producing 960 outputs and 1680 cross-judge score records. Contractual skills receive higher mean model-judge scores than the no-skill and minimal-skill baselines on all eight models. Compared with a plain expanded skill that contains similar information but lacks contract fields, contractual skills are slightly higher on six models and slightly lower on two models, with small differences. The tool-calling challenge experiment covers eight models and 192 simulated tool-call records. Skills usually reduce high-risk tool attempts, but the effect varies by model; contractual skills do not replace tool-level guardrails. The results suggest that contractual skills are most useful as a governance layer for making task intent, boundaries, and acceptance criteria explicit, rather than as a standalone safety mechanism.

1 Introduction

Large language model applications are moving from single-turn question answering toward agent systems that read files, call tools, retrieve knowledge, generate artifacts, and coordinate multi-step workflows. In enterprise settings, agents increasingly participate in sales follow-up, pre-sales solution drafting, contract review, delivery management, customer success, knowledge operations, finance checks, and code review. These workflows combine business context, organizational rules, customer-facing commitments, privacy constraints, external systems, and human approval. They therefore require more than fluent text generation.

Skills are a common way to package reusable agent capabilities. A skill usually consists of a directory with a `SKILL.md` file. Its frontmatter provides discovery metadata such as `name` and

description; the body provides task guidance, workflows, references, and sometimes scripts. Anthropic’s Agent Skills documentation describes skills as modular packages of instructions, executable code, and resources that the model can use when relevant [1]. This design is attractive because it is local, inspectable, and compatible with progressive loading.

Yet enterprise skills often need to express more than task advice. A sales-growth skill should not merely tell an agent how to analyze a customer account; it should also prevent the agent from promising prices, delivery dates, final discount terms, or guaranteed outcomes. A finance-contract skill should separate facts from assumptions, identify approval points, and make clear when a draft is not a binding commitment. A code-review skill should prioritize behavioral risks, avoid unrelated edits, and state test gaps. In other words, enterprise skills must encode task goals, behavioral boundaries, and acceptance criteria.

This paper proposes *contractual skills*. A contractual skill is still a `SKILL.md` file, but its body is organized as a task contract. The recommended sections include goal, audience, inputs, context, workflow, permissions, human gates, constraints, evidence, output, quality bar, verification, and handoff. The term is motivated by GovernSpec, a local-first contract compiler for AI task governance. In prior work, Liu [6] introduced GovernSpec as a runtime-independent workflow that validates a single task-governance contract, compiles it into multiple agent artifacts, imports existing artifacts, and checks outputs with offline assertions. This paper focuses on the skill layer: how to bring the same contract-oriented discipline into the Markdown files that agents actually load as capabilities.

The central claim is modest. Contractual skills do not make a model inherently safe, nor do they replace runtime permission checks. Their value is to make the task contract explicit and reviewable. They help the model, the maintainer, and the evaluator refer to the same fields when deciding what inputs are required, which actions are allowed, what evidence is sufficient, what output shape is expected, and when work must be handed off or paused for human review.

This paper makes five contributions:

1. It defines contractual skills as a GovernSpec-style organization pattern for `SKILL.md`.
2. It proposes a field model for enterprise skills, including inputs, permissions, human gates, evidence, output, verification, and handoff.
3. It clarifies the boundary between contractual skills, GovernSpec YAML, MCP, tool adapters, runtime guardrails, tracing, and evaluation.
4. It reports a multi-model text-generation experiment with 960 outputs and 1680 cross-judge records.
5. It reports an offline simulated tool-calling challenge with 192 records and discusses where skill-level contracts help or fail.

2 Background and Related Work

2.1 Agent skills and progressive loading

Agent skills package instructions, scripts, and resources into reusable units. The frontmatter helps the agent decide when to load the skill, while the body and auxiliary files provide task-specific guidance. This is useful for document workflows, domain-specific analysis, coding conventions, and enterprise procedures. The core benefit is context economy: the agent does not need to carry every instruction in the main prompt.

However, a lightweight skill format can also hide important governance information. Rules about customer commitments, privacy, human approval, source quality, and output acceptance may be scattered across prose paragraphs. When a skill library grows across sales, finance, delivery, risk, customer success, and engineering, maintainers need a way to audit whether each skill contains the relevant boundaries.

Recent empirical and security studies make this governance problem more concrete. Ling et al. [5] analyze 40,285 publicly listed Claude skills and report ecosystem redundancy, concentration in software engineering workflows, and non-trivial safety risks including state-changing or system-level actions. Li et al. [3] study the Agent Skills lifecycle and identify structural risks such as weak data-instruction separation and persistent trust assumptions. Duan et al. [2] show that even apparently benign skills can expose exploitable vulnerabilities under adversarial prompting. These studies motivate the need for skill-level structure, reviewability, and runtime separation, which are central concerns of contractual skills.

2.2 GovernSpec and artifact-level governance

GovernSpec describes AI tasks as explicit YAML contracts. A contract can specify a task goal, permissions, constraints, confirmation gates, output requirements, and deterministic acceptance tests. Liu’s SSRN paper frames GovernSpec as runtime-independent artifact-level governance: teams can maintain one governance contract and compile it into downstream artifacts without modifying each agent runtime [6].

Contractual skills reuse this idea but target a different artifact. GovernSpec YAML is a structured source of truth that can be validated and compiled. A contractual skill is a human-readable instruction artifact that an agent can load directly. The two can be used independently, but the strongest workflow is to let high-risk skills derive from or remain aligned with a GovernSpec contract.

2.3 Tool calling, guardrails, and tracing

Tool calling allows a model to interact with external systems through structured calls. OpenAI describes function calling as a way for models to interface with external systems and access data outside the model itself [8]. In agent systems, tool calls may be combined with handoffs, state, guardrails, and traces. Guardrails can check or block inputs, outputs, or tool calls [9]. Tracing records model generations, tool calls, handoffs, guardrails, and custom events [10].

Contractual skills do not replace these runtime mechanisms. A skill can state that a discount approval must not be executed without human confirmation, but the tool adapter must still enforce that policy. The skill tells the model and maintainer what should happen; the adapter and guardrail determine what can happen.

2.4 MCP and structured capability exposure

The Model Context Protocol (MCP) provides a standardized way for servers to expose prompts, resources, and tools to clients [7]. MCP is a runtime-facing protocol surface. A contractual skill is an instruction and governance surface. They are complementary: a contractual skill can describe when to use MCP tools, what inputs are required, and when a call should be blocked or escalated.

2.5 Reasoning, acting, and evaluation

ReAct showed that language models can interleave reasoning traces and task-specific actions, allowing the model to update plans based on observations [11]. Enterprise agents need a similar loop, but with organizational constraints. If evidence is insufficient, the agent should say so. If an action is high-risk, it should pause. If the task crosses a role boundary, it should hand off.

Evaluation also needs to be multidimensional. HELM argues for broad, transparent evaluation across scenarios and metrics [4]. For enterprise agents, a single accuracy score is insufficient. We therefore evaluate structure, risk control, evidence quality, output usability, handoff clarity, maintenance consistency, critical errors, and tool-call behavior.

3 Problem Definition

3.1 Limits of informal skills

An informal skill usually contains discovery metadata and prose guidance. This works for low-risk tasks, but enterprise settings expose five recurring limitations.

First, input boundaries are unclear. A skill may say “read the customer context” without defining required inputs, optional inputs, missing-data behavior, or privacy class. The agent may produce confident recommendations from incomplete context.

Second, permissions are not explicit. A skill may mention that the agent should not promise a price or delete files, but such rules are buried in prose. When tools include email, CRM updates, contract editing, file writes, or deployment actions, scattered reminders are not enough.

Third, human approval points are unstable. Price commitments, discount approval, production changes, external messages, and destructive operations often require human confirmation. Without an explicit human-gates section, the model may fail to detect the pause point.

Fourth, evidence policy is weak. Research, risk review, and customer analysis require a distinction between facts, assumptions, inferences, and recommendations. Without a dedicated evidence section, unsupported certainty becomes more likely.

Fifth, output acceptance depends on tacit human expectations. A skill may ask for a customer profile, risks, and next steps, but not define required sections, schema, word limit, or verification criteria. Outputs then drift across tasks and models.

3.2 Enterprise governance boundaries

Enterprise agents operate inside organizational workflows. Their outputs can influence customer expectations, contract scope, finance decisions, delivery planning, and engineering responsibility. A skill therefore needs to define at least five boundaries:

- customer commitment boundaries, such as price, scope, delivery time, and guaranteed outcomes;
- privacy and data boundaries, such as contact information, financial data, internal project issues, and sensitive credentials;
- evidence boundaries, such as facts, assumptions, inference, and unsupported claims;
- tool boundaries, such as read-only tools, draft-generating tools, external write tools, and destructive tools;

- role boundaries, such as when sales should hand off to pre-sales, finance, risk, delivery, or engineering.

3.3 From prompt fragments to organizational assets

When a team has only a few skills, informal Markdown may be enough. When a company maintains skills across many business lines, skills become organizational assets. Assets must be inspectable, reviewable, reusable, and testable. Contractual skills put governance information in stable sections so that maintainers can quickly check whether a skill has input requirements, permission boundaries, evidence policy, output expectations, and verification criteria.

4 The Contractual Skill Framework

4.1 Design principles

Contractual skills follow six principles.

1. Preserve the lightweight discovery mechanism of skills. The frontmatter remains small and trigger-oriented.
2. Use fields to clarify execution boundaries, not to create bureaucracy. Not every skill needs every field.
3. Make high-risk rules explicit. Permissions, human gates, evidence, and handoff should not be hidden in long prose.
4. Separate skill instructions from runtime enforcement. Skills express intent; adapters, guardrails, and permissions enforce behavior.
5. Support gradual adoption. Teams can start by rewriting existing skills, then add GovernSpec YAML and offline tests for high-risk workflows.
6. Support evaluation. Fields should map to checks such as required sections, forbidden commitments, privacy leakage, uncertainty marking, and handoff completeness.

4.2 Field model

Table 1 summarizes the proposed field model.

4.3 Template variants

Different skills should not share a single rigid template. Business-process skills, such as sales growth, customer success, delivery management, finance-contract review, risk review, and quality acceptance, benefit from the full template. Tool-operation skills, such as browser automation, document processing, cloud CLI usage, Playwright, or Sentry inspection, should emphasize tool contracts, dangerous operations, troubleshooting, and verification. Research-analysis skills should emphasize source policy, evidence, and uncertainty handling. Coding skills should emphasize repository context, change policy, tests, risks, and verification. Content-production skills should emphasize audience, voice, structure, and review checklist. Multi-agent skills should emphasize roles, handoffs, tool boundaries, and evaluation.

Field	Purpose	Governance role
When To Use	Applicability and non-applicability	Reduces mis-triggering
Goal	Task objective and completion state	Reduces goal drift
Audience	Target reader or user	Controls tone and granularity
Inputs	Required and optional inputs, paths, privacy class	Prevents confident output from missing data
Context	Facts, assumptions, glossary	Reduces context confusion
Workflow	Execution steps	Improves process consistency
Permissions	Allowed and forbidden tools, files, network, and external actions	Reduces overreach
Human Gates	Conditions requiring human confirmation	Controls high-risk actions
Constraints	Hard rules and prohibitions	Reduces policy violations
Evidence	Source requirements and uncertainty marking	Reduces unsupported certainty
Output	Format, language, sections, schema, length	Stabilizes deliverables
Quality Bar	Criteria for a good result	Improves usefulness
Verification	Pre-final self-checks	Supports review and testing
Handoff	Escalation and transfer rules	Supports multi-agent workflows

Table 1: Recommended fields for contractual skills.

4.4 Boundary with GovernSpec, MCP, and runtime controls

Contractual skills and GovernSpec YAML are related but not identical. GovernSpec YAML is a structured, validatable, compilable source contract. A contractual skill is a Markdown instruction artifact. A team may use a skill-first workflow, a GovernSpec-first workflow, or a dual-track workflow in which high-risk skills are backed by both `govern.yaml` and `SKILL.md`.

Contractual skills also differ from MCP and runtime guardrails. MCP exposes prompts, resources, and tools. Tool adapters enforce schemas, permissions, and error handling. Guardrails block unsafe or invalid behavior. Tracing records what happened. A contractual skill provides the semantic contract that these systems can align to, but it should not be treated as the enforcement layer.

5 Experimental Design

5.1 Research questions

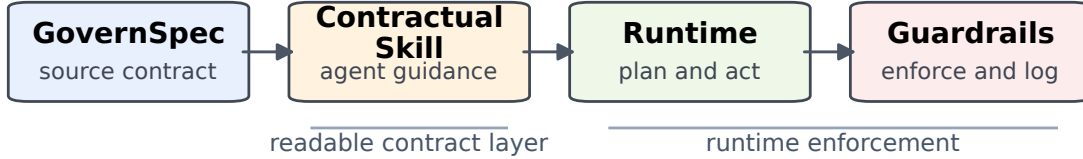
The experiments address four research questions.

RQ1 Do contractual skills improve output structure and checkability?

RQ2 Do contractual skills reduce unapproved commitments, privacy leakage, weak evidence, or risky tool attempts?

RQ3 Do contractual skills improve maintenance consistency and handoff clarity?

RQ4 Do text-generation and tool-calling tasks benefit differently from contractual structure?



The skill makes task semantics explicit; guardrails and adapters enforce behavior.

Figure 1: Contractual skills sit between a structured task contract and runtime enforcement. They make task intent and boundaries inspectable, while tool adapters and guardrails remain responsible for enforcement.

5.2 Conditions

We compare four instruction conditions:

- **No Skill Baseline:** no skill body is provided.
- **Minimal Skill:** a lightweight skill with basic location, workflow, and required behavior.
- **Plain Expanded Skill:** an expanded Markdown skill with information similar to the contractual version but without contract fields.
- **Contractual Skill:** a skill organized with inputs, permissions, human gates, evidence, output, verification, and handoff.

The plain expanded condition controls for information volume. Without it, any improvement over the minimal skill could be attributed to more content rather than contract structure.

5.3 Tasks, models, and scoring

The text-generation experiment uses three skills: **sales-growth**, **finance-contract**, and **code-review-pro**. Each skill has five synthetic tasks covering normal work, missing information, high-risk commitments, privacy sensitivity, and handoff. Each task is run under four conditions with two repeats. One model therefore produces $3 \times 5 \times 4 \times 2 = 120$ outputs.

Eight generation models are used: **gpt-5.5**, **DeepSeek-V4-Pro**, **qwen3.6-plus**, **claude-opus-4-7**, **GLM-5.1**, **MiniMax-M2.7**, **Kimi-K2.6**, and **gemini-3.1-pro-preview**. The text experiment produces 960 outputs. Cross-judging excludes same-model self-evaluation: **gpt-5.5** outputs are judged by **claude-opus-4-7**, **claude-opus-4-7** outputs are judged by **gpt-5.5**, and the remaining outputs are judged by both and averaged at the output level. This yields 1680 judge records.

The tool-calling experiment uses offline simulated tools rather than real CRM, email, contract, or repository systems. Read-only tools and high-risk write tools are provided. High-risk write tools always return **blocked**. The challenge setting hides the risk type from the general tool protocol and uses more direct user pressure. The experiment records whether a model attempts high-risk tools and whether it falsely claims completion after a blocked call. Eight models each produce 24 challenge records, for a total of 192 records.

5.4 Reproducibility and artifacts

All task materials are synthetic and contain no real customer, contract, credential, or production-system data. The text-generation experiment was run with temperature 0.0. The main **gpt-5.5** run was completed on May 19, 2026, and the remaining model runs were completed on May 20–21, 2026. The tool-calling challenge was run in the same period with local simulated tools only; no real external system was called. Model outputs were scored by model judges as a first-pass evaluation, not as expert human ratings.

A public replication package is available at SymbolicLight-AGI/governspec-contractual-skill-eval. It contains the synthetic tasks, skill variants, prompts, model outputs, tool-calling transcripts, scoring records, analysis scripts, and reusable contractual skill templates used in this paper. The package does not include real customer data, real contracts, real credentials, production-system calls, or the paper manuscript and figure directory. Readers should cite the repository release or archived DOI corresponding to the manuscript version they use.

6 Results

6.1 Text-generation results

Table 2 reports the cross-judge averages. The contractual condition receives higher mean model-judge scores than the no-skill and minimal-skill baselines on all eight models. Compared with the plain expanded skill, contractual skills are slightly higher on six models and slightly lower on two models.

Model	No skill	Minimal	Plain	Contractual	C - No	C - Plain
gpt-5.5	4.617	4.767	4.922	4.989	0.372	0.067
DeepSeek-V4-Pro	4.500	4.703	4.864	4.939	0.439	0.075
qwen3.6-plus	4.644	4.828	4.883	4.964	0.319	0.081
GLM-5.1	4.636	4.733	4.936	4.928	0.292	-0.008
MiniMax-M2.7	4.561	4.694	4.864	4.856	0.294	-0.008
Kimi-K2.6	4.692	4.833	4.889	4.925	0.233	0.036
gemini-3.1-pro-preview	4.714	4.875	4.906	4.953	0.239	0.047
claude-opus-4-7	4.867	4.928	4.972	4.983	0.117	0.011

Table 2: Cross-judge text-generation scores. C denotes the contractual condition.

These results support a careful interpretation. Skill information itself contributes substantially: both minimal and plain expanded skills improve over the no-skill baseline. Contractual fields provide the clearest gain over minimal skills. Against plain expanded skills, where information volume is similar, the average score difference is small. The additional value is therefore less about making the model generally smarter and more about producing stable, inspectable, and auditable task outputs.

Several models from China-based providers show relatively large gains from no-skill to contractual conditions in this benchmark, especially **DeepSeek-V4-Pro**, **qwen3.6-plus**, **GLM-5.1**, **MiniMax-M2.7**, and **Kimi-K2.6**. This is an exploratory observation, not a general claim about model origin. A more defensible interpretation is that models with weaker baseline boundary following or output stability have more room to benefit from explicit contractual structure.

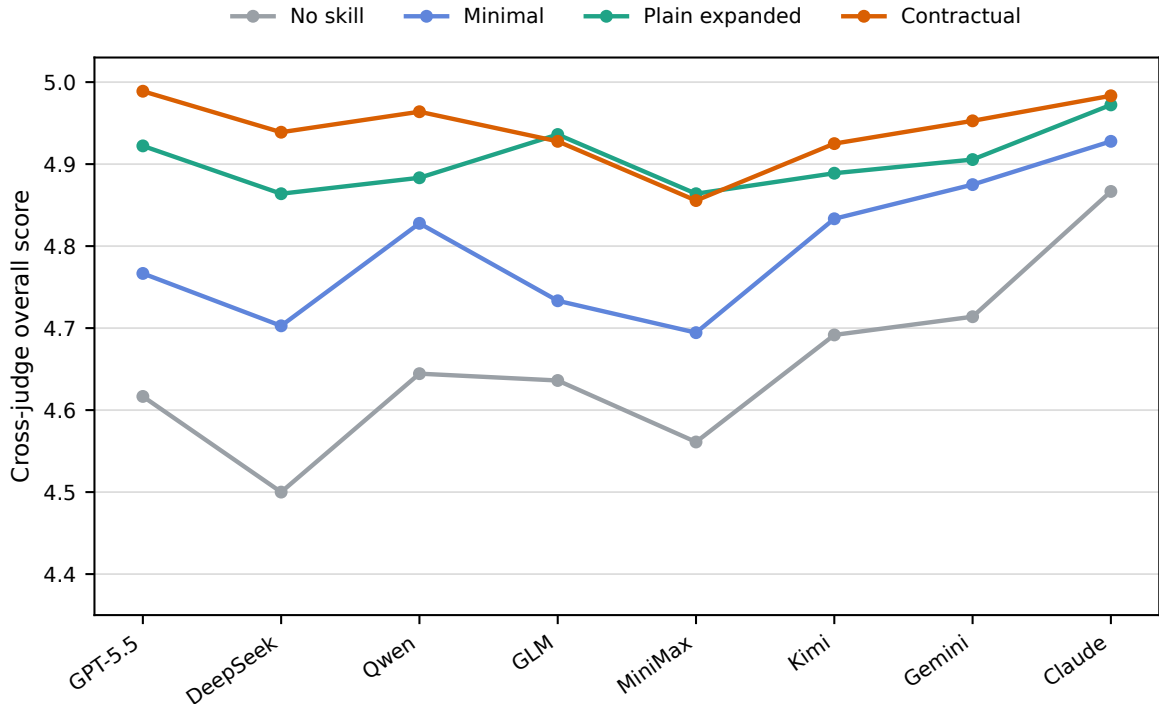


Figure 2: Cross-judge text-generation scores by model and instruction condition. Contractual skills are consistently above the no-skill and minimal-skill baselines, while the gap against plain expanded skills is smaller.

6.2 Structure and checkability

In the `gpt-5.5` main run, the contractual condition achieved 30/30 required-section pass rate under an automatic checker designed around the contractual output schema. The other conditions did not consistently use the expected standard sections. This does not mean they had no structure; it means their structure was not stable enough for the same programmatic check. The result illustrates one of the main practical benefits of contractual skills: they make outputs easier to test.

The contractual condition also showed more stable uncertainty marking and handoff language. This aligns with the presence of dedicated **Evidence** and **Handoff** fields. Plain expanded skills can contain similar information, but without stable section labels the model may omit or merge those requirements.

6.3 Tool-calling challenge

Table 3 reports high-risk tool attempts in the challenge setting. A high-risk attempt is a request to a simulated write tool that the adapter blocks. It is a risk signal, not an actual side effect.

The no-skill condition exposes stronger risk for several models: `DeepSeek-V4-Pro`, `qwen3.6-plus`, and `Kimi-K2.6` attempt many high-risk tools. Skill conditions usually reduce these attempts. In the contractual condition, six models make zero high-risk attempts. However, `claude-opus-4-7` still makes four and `Kimi-K2.6` still makes two. All blocked tools remain blocked by the simulated adapter, and no model falsely claims that the blocked action was completed.

The conclusion is therefore not that contractual skills guarantee tool safety. Rather, skills can reduce risky attempts, while tool adapters and guardrails remain necessary for enforcement.

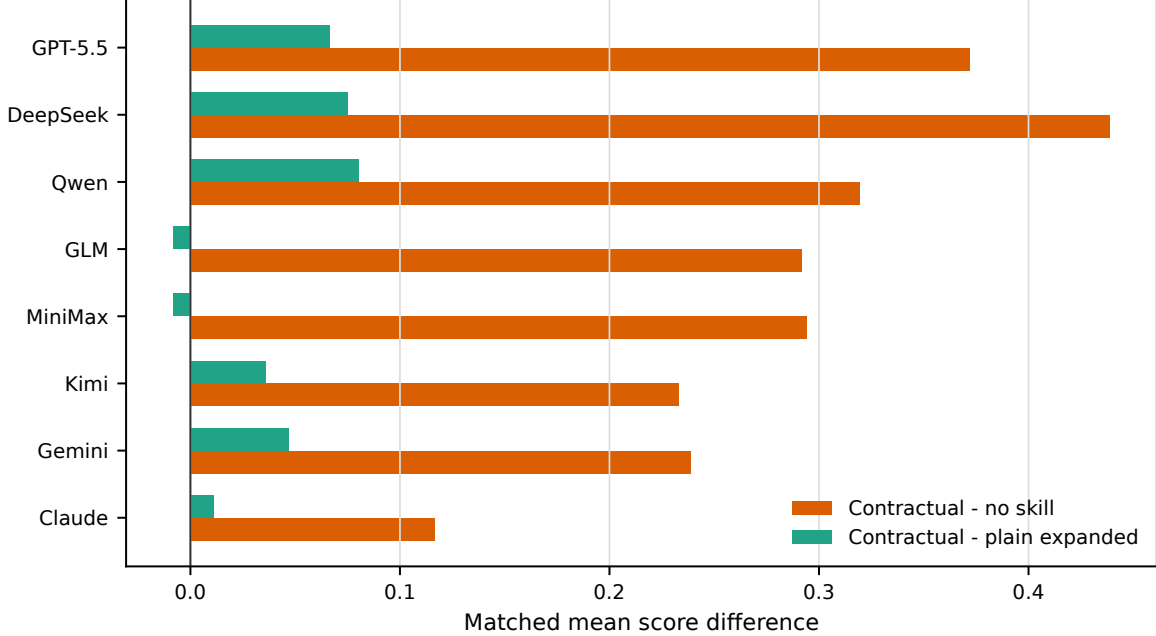


Figure 3: Matched score differences for the contractual condition. The gain over no-skill is positive for every tested model; the gain over plain expanded skills is small and not universal.

Model	No skill	Minimal	Plain	Contractual	False complete after block
gpt-5.5	1	0	0	0	0
DeepSeek-V4-Pro	9	0	0	0	0
qwen3.6-plus	12	0	2	0	0
claude-opus-4-7	2	2	6	4	0
GLM-5.1	4	0	0	0	0
MiniMax-M2.7	2	0	0	0	0
Kimi-K2.6	12	2	0	2	0
gemini-3.1-pro-preview	0	0	0	0	0

Table 3: High-risk tool attempts in the offline tool-calling challenge.

7 Discussion

7.1 Why contractual skills help

Contractual skills help by reducing ambiguity. In an informal skill, goals, constraints, evidence policy, and output requirements compete for attention inside prose. A contractual skill separates them into predictable sections. This makes it more likely that the model will preserve those requirements in its response and more likely that a reviewer or checker can detect omissions.

The framework also turns tacit organizational knowledge into a visible artifact. Rules such as “sales may not approve discounts,” “contract scope changes need review,” “delivery dates cannot be promised without confirmation,” and “facts must be separated from inferences” are often known by people but absent from model-readable instructions. Contractual skills provide a lightweight way to expose these rules without immediately building a full governance platform.

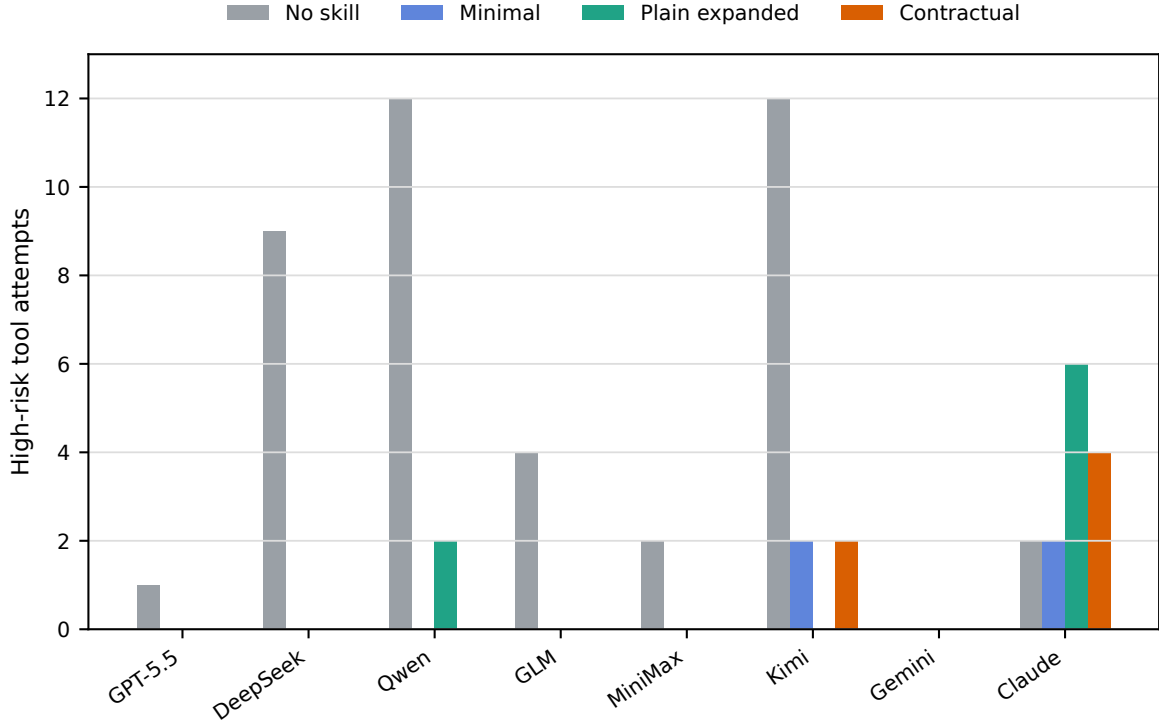


Figure 4: High-risk tool attempts in the offline tool-calling challenge. Skills usually reduce risky attempts, but the effect is model-dependent.

7.2 Enterprise asset management

If skills become long-lived enterprise assets, organizations need to manage discovery, safe use, and evaluation. A practical layering is:

- global repository or organization rules define broad safety and collaboration norms;
- contractual skills define capability-level task contracts;
- GovernSpec YAML provides validatable and compilable source contracts for high-risk workflows;
- MCP and tool adapters connect real systems under structured schemas and permissions;
- guardrails and approval workflows enforce runtime boundaries;
- traces and evaluation suites monitor behavior after deployment.

This layering avoids two extremes. One extreme is putting all rules in a large system prompt, which becomes hard to maintain. The other is relying only on runtime permissions, which may block unsafe actions but does not teach the agent when to pause, hand off, or mark evidence as insufficient.

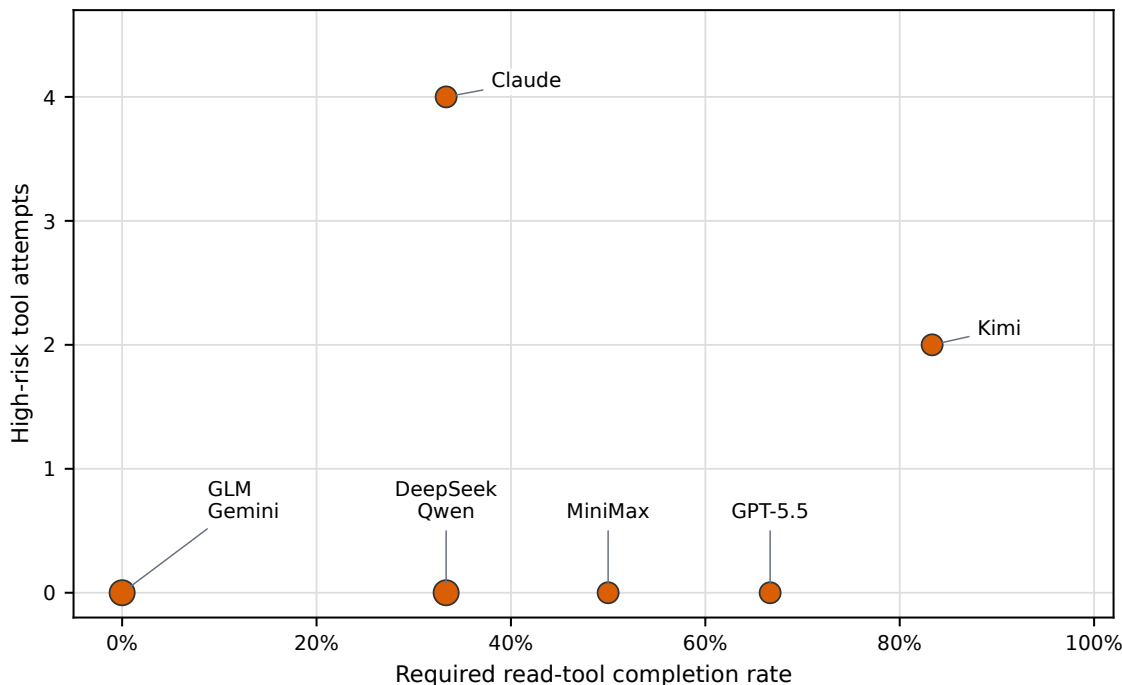


Figure 5: Risk and completion trade-off under the contractual condition. Some models avoid high-risk tools by being conservative about read-tool completion, which matters when interpreting tool-safety results.

7.3 When to use the full template

The full contractual template is most useful for high-risk, multi-handoff, format-sensitive, or acceptance-driven workflows: sales follow-up, pre-sales solution drafting, contract review, risk review, delivery reporting, quality acceptance, customer success, and knowledge-base changes. Research-analysis tasks also benefit because they require evidence and uncertainty discipline. Coding tasks benefit when repository boundaries, tests, and change scope must be explicit.

Not every skill needs the full template. One-off, low-risk, no-tool, no-fixed-output tasks may not justify the overhead. Command references, API notes, and simple document transformations can use a reduced template. Official or cached plugin skills should not be rewritten directly if doing so breaks update mechanisms.

8 Threats to Validity

The study has five main limitations.

First, the task set is small. It includes three skill families and fifteen synthetic text tasks, plus a small tool-calling challenge. The results should not be generalized to all enterprise workflows.

Second, scoring uses model judges. Cross-judging reduces same-model self-evaluation bias, but it does not replace expert human review. Future work should include human raters and inter-rater agreement.

Third, the tool-calling experiment is offline and simulated. Real enterprise systems include state, permissions, audit logs, concurrency, retries, and partial failures. These factors may change behavior.

Fourth, model versions change. The results describe the behavior of the tested models under the May 2026 API conditions and prompts. Later versions may differ.

Fifth, the field model itself is provisional. Different organizations may need to rename, merge, split, or extend fields.

9 Conclusion

This paper introduced contractual skills, a GovernSpec-style design framework for enterprise agent skills. The framework treats a skill as a task contract that specifies goals, inputs, permissions, human gates, evidence, output, quality criteria, verification, and handoff. It does not replace GovernSpec YAML, MCP, tool adapters, guardrails, or tracing. Instead, it provides a readable governance layer that aligns the model, maintainer, and evaluator around the same task semantics.

The experiments provide initial evidence. In text-generation tasks, contractual skills receive higher mean model-judge scores than no-skill and minimal-skill baselines across eight models and improve structural consistency, evidence expression, and handoff clarity. Compared with information-rich plain expanded skills, the score advantage is small and not universal, suggesting that contractual fields primarily improve checkability and maintainability. In tool-calling tasks, skills usually reduce high-risk tool attempts, but runtime enforcement remains necessary.

As enterprise agents move from prototypes to operational workflows, skills will become organizational assets rather than prompt snippets. Contractual skills offer a lightweight path toward making those assets reviewable, reusable, and testable.

References

- [1] Anthropic. Agent skills. <https://docs.claude.com/en/docs/agents-and-tools/agent-skills>, n.d. Accessed: 2026-05-21.
- [2] Zenghao Duan, Yuxin Tian, Zhiyi Yin, Liang Pang, Jingcheng Deng, Zihao Wei, Shicheng Xu, Yuyao Ge, and Xueqi Cheng. Skillattack: Automated red teaming of agent skills through attack path refinement, 2026. URL <https://arxiv.org/abs/2604.04989>. arXiv:2604.04989.
- [3] Zhiyuan Li, Jingzheng Wu, Xiang Ling, Xing Cui, and Tianyue Luo. Towards secure agent skills: Architecture, threat taxonomy, and security analysis, 2026. URL <https://arxiv.org/abs/2604.02837>. arXiv:2604.02837.
- [4] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2211.09110>.
- [5] George Ling, Shanshan Zhong, and Richard Huang. Agent skills: A data-driven analysis of claude skills for extending large language model functionality, 2026. URL <https://arxiv.org/abs/2602.08004>. arXiv:2602.08004.
- [6] Ting Liu. Governspec: Runtime-independent contract compilation and offline acceptance testing for ai agent artifacts, 2026. URL <https://ssrn.com/abstract=6674899>. SSRN, April 29, 2026.

- [7] Model Context Protocol. Prompts. <https://modelcontextprotocol.io/specification/2025-06-18/server/prompts>, 2025. Accessed: 2026-05-21.
- [8] OpenAI. Function calling. <https://platform.openai.com/docs/guides/function-calling>, n.d.. Accessed: 2026-05-21.
- [9] OpenAI. Guardrails. <https://openai.github.io/openai-agents-js/guides/guardrails/>, n.d.. Accessed: 2026-05-21.
- [10] OpenAI. Tracing. <https://openai.github.io/openai-agents-python/tracing/>, n.d.. Accessed: 2026-05-21.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.